

Activity 1 — Task 4: Continuous Salary Regression Modeling

Group 08

2026-08-24

Contents

4 Continuous Salary Regression Modeling	2
4.1 Training and testing partitions	2
4.2 Multiple Linear Regression	4
4.3 Polynomial Regression	14
4.4 Ridge and LASSO Regression	15
4.5 Summary of training-set fit	20

This section addresses the first research question: *how accurately can we predict the salary of a Data Science professional worldwide, and which factors drive income levels?* We model `salary_in_usd` through its natural logarithm `log_salary`, because the exploratory analysis showed a strongly right-skewed distribution with a handful of legitimate executive-level salaries; working on the log scale accommodates that skew without discarding real observations, and it gives coefficients a convenient percentage interpretation.

Seven model specifications are constructed and their assumptions tested: a **multiple linear regression**, first with all available predictors and then reduced by stepwise selection; a **polynomial regression** that lets the numeric predictors bend away from a straight line; **Ridge**, **LASSO**, and their **elastic-net** mixture with a cross-validated mixing weight, which stabilize estimates when many dummy variables share the workload; and a final linear model fitted on the Box–Cox-optimal power transform `salaryλ`. All fitting, tuning, and assumption testing use the training data only; the held-out records are kept untouched here and are formally scored and compared in Task 6.

```
library(tidyverse)
library(glmnet)
library(car)
library(lmtest)
library(MASS)
library(tseries)      # jarque.bera.test
library(sandwich)     # HC-robust standard errors
library(broom)        # tidy coefficient tables

data <- read.csv("dataset/ds_salaries_clean.csv")

set.seed(2026)        # fixed seed: every randomized step below is reproducible
```

4 Continuous Salary Regression Modeling

4.1 Training and testing partitions

Model quality is judged on observations not seen during fitting, so we split the 565 cleaned records into training (80%) and test (20%) sets. The first research question asks how accurately salaries can be predicted **globally** and which factors drive income across the whole sample, so every observation era has an equal chance of being held out: a single random 80 : 20 split of the 565 records assigns 452 rows to training and 113 to test, preserving the global mix of years, roles, and locations in both partitions. No temporal or year-aware constraint is imposed, because the goal is a model for the population of data-science salaries as a whole, not a forecast of any single future year.

The draw is checked so that rare categories (contract and freelance employment have 5 and 4 records dataset-wide) stay represented in training; the fixed seed makes the split fully reproducible.

```
used_cols <- c("log_salary", "salary_in_usd", "experience_level", "employment_type",
               "company_size", "job_category", "location_group", "remote_ratio",
               "is_local", "work_year",
               "exp_level_ord_scaled", "company_size_ord_scaled", "remote_ratio_scaled")
stopifnot(nrow(data) == 565, sum(is.na(data[, intersect(used_cols, names(data))])) == 0)

# Factor baselines chosen for interpretability: coefficients then read as
# contrasts against the common category (FT employment, Data Analyst,
# rest-of-world location).
data$experience_level <- factor(data$experience_level, levels = c("EN", "MI", "SE", "EX"))
data$company_size <- factor(data$company_size, levels = c("S", "M", "L"))
data$employment_type <- relevel(factor(data$employment_type), ref = "FT")
data$job_category <- relevel(factor(data$job_category), ref = "Data_Analyst")
data$location_group <- relevel(factor(data$location_group), ref = "Other")

data <- data %>% mutate(row_id = row_number())

# Plain random 80:20 split across the whole dataset (global model, seed 2026).
# Rare categories must stay represented in training, so the draw is re-tried
# until every test-category level also appears among the training rows.
check_vars <- c("employment_type", "job_category", "location_group",
               "experience_level", "company_size")

repeat {
  idx <- sample(nrow(data), floor(0.8 * nrow(data)))
  tr <- data[idx, ]
  te <- data[-idx, ]
  ok <- all(vapply(check_vars,
                   function(v) all(unique(te[[v]]) %in% unique(tr[[v]])), logical(1)))
  if (ok) break
}
train <- tr
test <- te

stopifnot(nrow(train) == 452, nrow(test) == 113,
          all(!test$row_id %in% train$row_id))

write.csv(train, "dataset/train_data.csv", row.names = FALSE)
write.csv(test, "dataset/test_data.csv", row.names = FALSE)
```

Both partitions preserve a broad global mix of years, experience levels, job families, and employment types. The rare CT category appears only in training under this seed, while every category observed in the test set is represented in training, as the following composition tables show:

```
table(train$work_year);      table(test$work_year)
```

```
##
## 2020 2021 2022
##    60   164   228
```

```
##
## 2020 2021 2022
##    12    51    50
```

```
table(train$experience_level); table(test$experience_level)
```

```
##
##  EN  MI  SE  EX
##  73 163 195  21
```

```
##
##  EN  MI  SE  EX
##  15  45  48   5
```

```
table(train$job_category);    table(test$job_category)
```

```
##
##   Data_Analyst  Data_Engineer  Data_Scientist  Management_Lead  ML_AI_Engineer      Other
##              83             119             113              46             65         12
##      Research
##              14
```

```
##
##   Data_Analyst  Data_Engineer  Data_Scientist  Management_Lead  ML_AI_Engineer      Other
##              18              32              30              12             16         3
##      Research
##              2
```

```
table(train$employment_type); table(test$employment_type)
```

```
##
##  FT  CT  FL  PT
## 437   5   3   7
```

```
##
##  FT  CT  FL  PT
## 109   0   1   3
```

4.2 Multiple Linear Regression

We build the regression the way an analyst should: start from the simplest defensible choice — an ordinary linear model on the **raw dollar salary** — confront it with the classical assumptions, let a Box–Cox analysis select a better response scale, and then verify that the corrected model genuinely behaves better. Only after the response scale is settled do we refine the model and interpret its coefficients.

A reference model on raw dollars

All models in this section share the same right-hand side — experience level, job category, employment type, company size, grouped employer location, remote-work ratio, and survey year. The reference version regresses the raw dollar amount on them:

```
ols_raw <- lm(salary_in_usd ~ experience_level + job_category + employment_type +
              company_size + location_group + remote_ratio + work_year,
              data = train)
summary(ols_raw)
```

```
##
## Call:
## lm(formula = salary_in_usd ~ experience_level + job_category +
##     employment_type + company_size + location_group + remote_ratio +
##     work_year, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -128072  -30464   -4132   18197  352514
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.043e+07  9.296e+06   1.122  0.262373
## experience_levelMI  1.910e+04  8.366e+03   2.283  0.022916 *
## experience_levelSE  4.546e+04  8.758e+03   5.191  3.24e-07 ***
## experience_levelEX  1.215e+05  1.517e+04   8.008  1.11e-14 ***
## job_categoryData_Engineer  2.981e+04  8.040e+03   3.707  0.000237 ***
## job_categoryData_Scientist  3.181e+04  8.101e+03   3.927  0.000100 ***
## job_categoryManagement_Lead  3.249e+04  1.102e+04   2.949  0.003363 **
## job_categoryML_AI_Engineer  4.063e+04  9.648e+03   4.211  3.10e-05 ***
## job_categoryOther  2.541e+04  1.772e+04   1.434  0.152264
## job_categoryResearch  5.197e+04  1.652e+04   3.145  0.001774 **
## employment_typeCT  4.853e+04  2.521e+04   1.925  0.054940 .
## employment_typeFL -8.998e+04  3.239e+04  -2.778  0.005714 **
## employment_typePT -4.175e+04  2.198e+04  -1.899  0.058205 .
## company_sizeM  1.450e+04  8.930e+03   1.624  0.105199
## company_sizeL  2.773e+04  8.419e+03   3.294  0.001069 **
## location_groupCA  3.758e+04  1.392e+04   2.699  0.007220 **
## location_groupDE  1.919e+04  1.474e+04   1.302  0.193706
## location_groupES -6.524e+03  1.917e+04  -0.340  0.733838
## location_groupFR  6.671e+03  1.725e+04   0.387  0.699127
## location_groupGB  2.566e+04  1.175e+04   2.184  0.029474 *
## location_groupIN -2.690e+04  1.363e+04  -1.974  0.049078 *
## location_groupUS  7.752e+04  8.293e+03   9.347  < 2e-16 ***
## remote_ratio    5.393e+01  6.717e+01   0.803  0.422538
```

```
## work_year          -5.170e+03  4.601e+03  -1.124 0.261818
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 54930 on 428 degrees of freedom
## Multiple R-squared:  0.4893, Adjusted R-squared:  0.4619
## F-statistic: 17.83 on 23 and 428 DF,  p-value: < 2.2e-16
```

On its own scale the model is respectable at first glance (adjusted $R^2 = 0.462$), but fit quality is not what concerns us — the assumptions are.

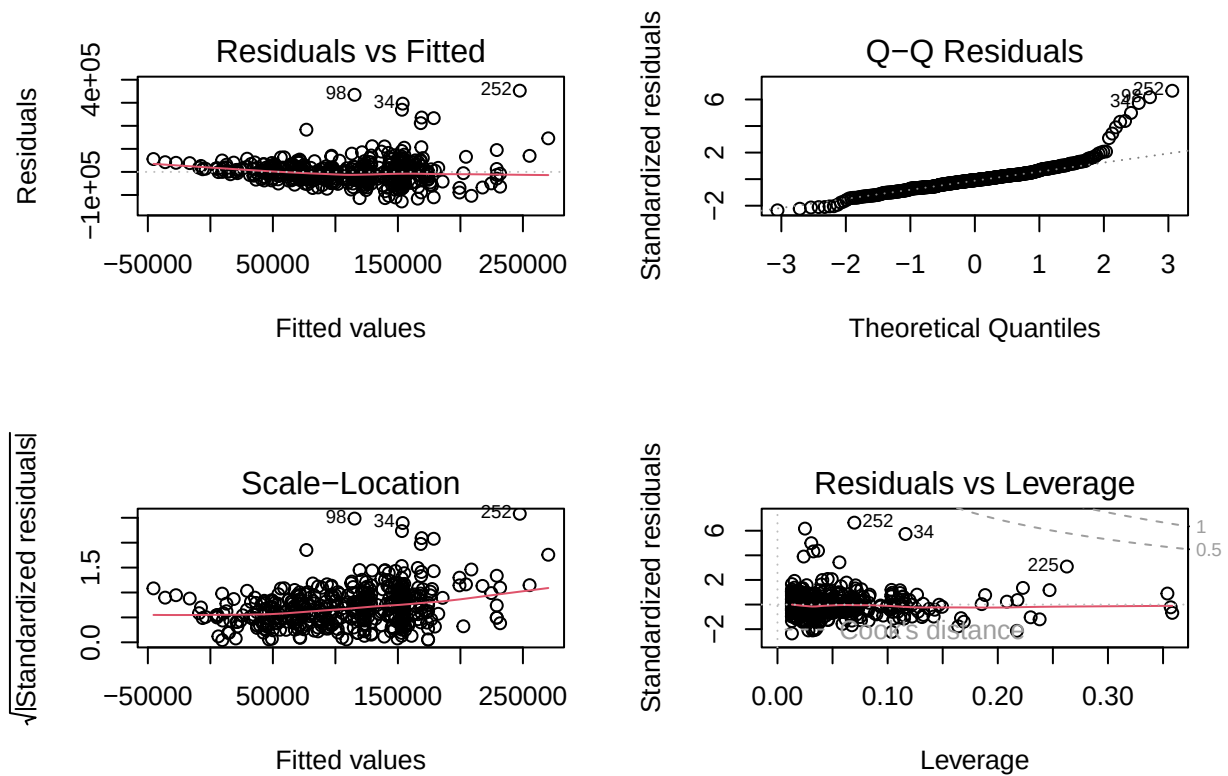
Diagnostic checking of the reference model

The four base residual plots, formal normality tests, a Breusch–Pagan test for homoskedasticity, and Cook’s distance (influence of single observations) are applied so that every assumption is *tested*, never assumed.

```
diag_row <- function(label, mod) {
  sw <- shapiro.test(resid(mod))
  jb <- tseries::jarque.bera.test(resid(mod))
  bp <- lmtest::bptest(mod)
  tibble(response      = label,
          Shapiro_p     = signif(sw$p.value, 3),
          Jarque_Bera_p = signif(jb$p.value, 3),
          BP_stat       = round(unnamed(bp$statistic), 1),
          BP_p          = signif(bp$p.value, 3),
          Max_Cooks_D   = round(max(cooks.distance(mod)), 3))
}
diag_row <- diag_row("salary_in_usd", ols_raw)
diag_row
```

response	Shapiro_p	Jarque_Bera_p	BP_stat	BP_p	Max_Cooks_D
salary_in_usd	0	0	41.6	0.0101	0.18

```
par(mfrow = c(2, 2)); plot(ols_raw); par(mfrow = c(1, 1))
```

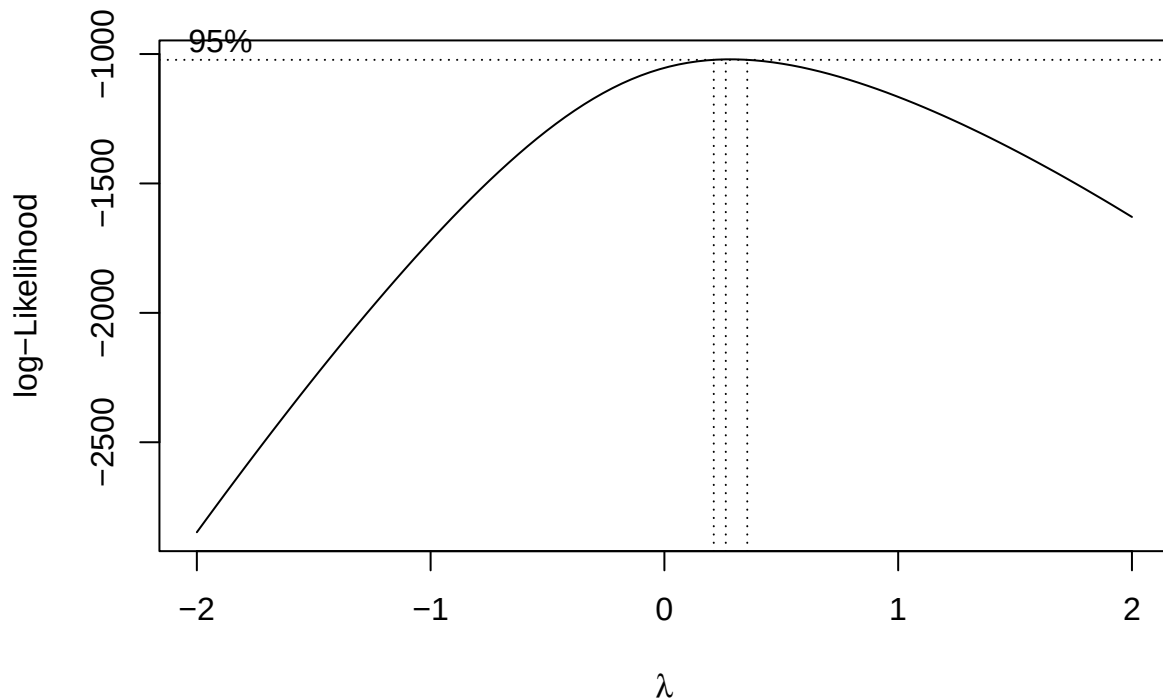


Every check fails. Residual normality is rejected at astronomical significance ($p = 1.3 \cdot 10^{21}$), errors are heteroskedastic (BP = 41.6, $p = 0.010$), and the largest Cook's distance reaches 0.18 — a handful of executive-level records visibly dominate the fitted line. An untransformed linear model is therefore not a trustworthy basis for inference or prediction.

Choosing the response scale: Box-Cox analysis

The Box-Cox family asks which power λ makes the linear model most consistent with its assumptions:

```
bc <- MASS::boxcox(ols_raw, plotit = TRUE)
```



```
lambda_hat <- bc$x[which.max(bc$y)]
## Profile-likelihood 95% confidence interval for lambda
cut_off <- max(bc$y) - qchisq(0.95, 1) / 2
lambda_ci <- range(bc$x[bc$y >= cut_off])
cat("Box-Cox optimal lambda:", round(lambda_hat, 3), "\n")
```

```
## Box-Cox optimal lambda: 0.263
```

```
cat("95% CI: [", round(lambda_ci[1], 3), ",", round(lambda_ci[2], 3), "]\n")
```

```
## 95% CI: [ 0.222 , 0.343 ]
```

The likelihood peaks at $\hat{\lambda} \approx 0.26$, with a 95% profile-likelihood interval $[0.22, 0.34]$ that excludes **both** special cases: $\lambda = 1$ (raw dollars, rejected above) and $\lambda = 0$ (the log). Formally, the data prefer an intermediate power. We adopt $\ln(\text{salary})$ as the working response for its interpretable percentage coefficients, and give $\text{salary}^{0.26}$ the identical battery of assumption tests below; whether the intermediate power rivals $\ln(\text{salary})$ out of sample is deferred to Task 6, which owns the held-out comparison.

The regression on $\ln(\text{salary})$

$$\ln(\text{salary}_i) = \beta_0 + \beta_1 \text{experience}_i + \beta_2 \text{role}_i + \beta_3 \text{contract type}_i + \beta_4 \text{company size}_i + \beta_5 \text{location}_i + \beta_6 \text{remote ratio}_i + \beta_7 \text{year}_i + \varepsilon_i$$

Location enters through the grouped employer-country variable only. The employee-residence dummies are deliberately left out: for the largely non-remote workforce the two countries coincide, and carrying both would introduce near-duplicate predictors whose inflated variances distort every coefficient's standard error.

```
ols_full <- lm(log_salary ~ experience_level + job_category + employment_type +
               company_size + location_group + remote_ratio + work_year,
               data = train)
summary(ols_full)
```

```
##
## Call:
## lm(formula = log_salary ~ experience_level + job_category + employment_type +
##     company_size + location_group + remote_ratio + work_year,
##     data = train)
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
	-2.79400	-0.21402	0.02879	0.29527	1.69508

```
##
## Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-1.383e+01	8.415e+01	-0.164	0.869567
experience_levelMI	2.184e-01	7.573e-02	2.884	0.004127 **
experience_levelSE	5.130e-01	7.928e-02	6.471	2.67e-10 ***
experience_levelEX	9.454e-01	1.373e-01	6.885	2.07e-11 ***
job_categoryData_Engineer	2.700e-01	7.278e-02	3.710	0.000234 ***
job_categoryData_Scientist	3.089e-01	7.333e-02	4.213	3.08e-05 ***
job_categoryManagement_Lead	4.346e-01	9.973e-02	4.358	1.65e-05 ***
job_categoryML_AI_Engineer	3.718e-01	8.734e-02	4.257	2.55e-05 ***
job_categoryOther	1.554e-01	1.604e-01	0.969	0.333323
job_categoryResearch	4.884e-01	1.496e-01	3.266	0.001179 **
employment_typeCT	2.465e-01	2.282e-01	1.080	0.280754
employment_typeFL	-1.284e+00	2.932e-01	-4.378	1.51e-05 ***
employment_typePT	-1.179e+00	1.990e-01	-5.927	6.37e-09 ***
company_sizeM	1.873e-01	8.083e-02	2.317	0.020972 *
company_sizeL	3.185e-01	7.621e-02	4.179	3.55e-05 ***
location_groupCA	6.665e-01	1.260e-01	5.289	1.97e-07 ***
location_groupDE	5.725e-01	1.334e-01	4.290	2.21e-05 ***
location_groupES	1.014e-01	1.736e-01	0.584	0.559223
location_groupFR	3.409e-01	1.561e-01	2.183	0.029576 *
location_groupGB	5.460e-01	1.063e-01	5.136	4.28e-07 ***
location_groupIN	-6.680e-01	1.234e-01	-5.413	1.03e-07 ***
location_groupUS	9.495e-01	7.507e-02	12.648	< 2e-16 ***
remote_ratio	6.138e-04	6.081e-04	1.009	0.313365
work_year	1.174e-02	4.165e-02	0.282	0.778161

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4972 on 428 degrees of freedom
## Multiple R-squared:  0.6383, Adjusted R-squared:  0.6188
## F-statistic: 32.84 on 23 and 428 DF,  p-value: < 2.2e-16
```

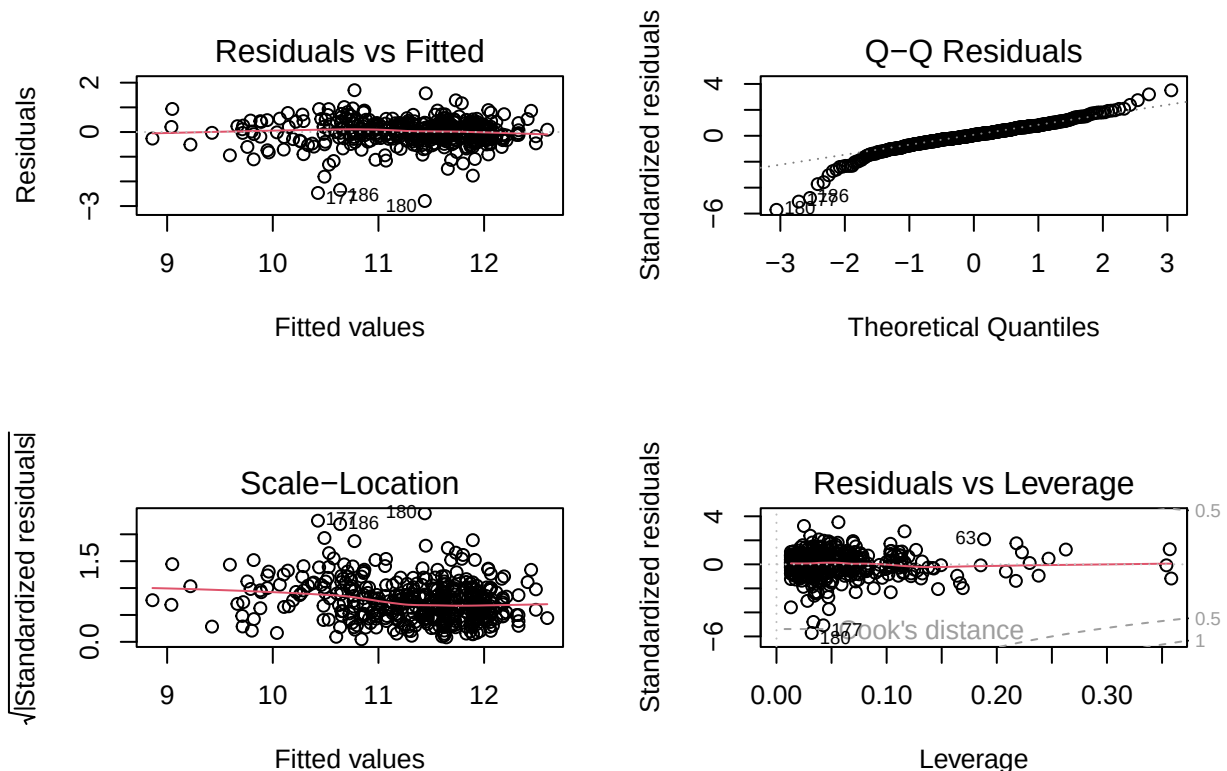
The transformed model explains about **62% of the variation in log-salary** (adjusted $R^2 = 0.619$) and is highly significant overall ($F = 32.8$ on 23 and 428 df, $p < 10^{-1}$) — a clear gain over the raw-scale reference.

Re-checking the assumptions after the transformation

```
## The Box-Cox-optimal power scale receives the identical battery of tests,
## so the transformation choice is judged on evidence rather than habit.
z_train <- train$salary_in_usd^lambda_hat
ols_power <- lm(z_train ~ experience_level + job_category + employment_type +
               company_size + location_group + remote_ratio + work_year,
               data = train)
diag_power <- diag_row(paste0("salary^", round(lambda_hat, 2)), ols_power)
diag_log <- diag_row("ln(salary)", ols_full)
bind_rows(diag_raw, diag_power, diag_log)
```

response	Shapiro_p	Jarque_Bera_p	BP_stat	BP_p	Max_Cooks_D
salary_in_usd	0	0	41.6	0.0101	0.180
salary^0.26	0	0	34.6	0.0571	0.076
ln(salary)	0	0	40.9	0.0121	0.048

```
par(mfrow = c(2, 2)); plot(ols_full); par(mfrow = c(1, 1))
```



```
## Generalized VIFs: a property of the predictor design matrix, hence
## identical under any response scale - one table covers both models.
vif_raw <- car::vif(ols_full)
```

```

if (is.matrix(vif_raw)) {
  vif_tab <- tibble(
    term      = rownames(vif_raw),
    GVIF      = vif_raw[, "GVIF"],
    Df        = vif_raw[, "Df"],
    gvif_adj  = vif_raw[, "GVIF^(1/(2*Df))"]
  )
} else {
  vif_tab <- tibble(term = names(vif_raw), GVIF = as.numeric(vif_raw),
                    Df = 1, gvif_adj = sqrt(as.numeric(vif_raw)))
}
vif_tab

```

term	GVIF	Df	gvif_adj
experience_level	1.720455	3	1.094647
job_category	1.841902	6	1.052218
employment_type	1.189774	3	1.029384
company_size	1.599767	2	1.124642
location_group	1.838258	7	1.044446
remote_ratio	1.117734	1	1.057229
work_year	1.582721	1	1.258062

Side by side, the log transformation earns its place: the Shapiro–Wilk p -value rises from $1.3 \cdot 10^{-21}$ (raw dollars) to $1.9 \cdot 10^{-1}$ — a roughly fourteen-million-fold improvement in consistency with normality — and the maximum Cook’s distance falls by a factor of nearly four ($0.180 \rightarrow 0.048$), so no single record any longer dictates the fit. The Box–Cox-optimal power does better still: normality improves to $p = 1.8 \cdot 10^{-4}$ and its Breusch–Pagan statistic (34.6, $p = 0.057$) is no longer significant, unlike the log scale’s (40.9, $p = 0.012$) and the raw scale’s (41.6, $p = 0.010$). Two conclusions follow. First, the log transform clearly repairs the worst failures of the raw-dollar model, but residual non-normality (and on the log scale, heteroskedasticity) persists — the rejections trace to the thin cluster of very low salaries documented in the EDA — so robust inference is required. Second, because the power scale dominates on diagnostics and even reaches homoskedasticity, keeping $\ln(\text{salary})$ as the working response rests on its interpretable percentage coefficients (used throughout this section); Task 6 holds the empirical verdict on which scale predicts best. On the collinearity side, all df-adjusted GVIFs lie between 1.03 and 1.26 — far under any common threshold — confirming that the single location route avoided redundancy.

Refining the model by AIC

The full model contains two linear trend terms (`remote_ratio`, `work_year`) whose usefulness a parsimonious model should justify. Stepwise selection in both directions, guided by Akaike’s information criterion, removes exactly those two terms while keeping all five factor blocks: the AIC improves from 676 to 674 at essentially unchanged adjusted R^2 ($0.619 \rightarrow 0.620$), i.e., the simpler model predicts equally well with two fewer terms.

```

ols_step <- step(ols_full, direction = "both", trace = FALSE)
summary(ols_step)

```

```

##
## Call:
## lm(formula = log_salary ~ experience_level + job_category + employment_type +
##      company_size + location_group, data = train)

```

```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.77343 -0.21545  0.02545  0.27967  1.63835
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      9.93811    0.10626   93.526 < 2e-16 ***
## experience_levelMI  0.21512    0.07522    2.860 0.004448 **
## experience_levelSE  0.51724    0.07794    6.636 9.72e-11 ***
## experience_levelEX  0.95315    0.13608    7.004 9.62e-12 ***
## job_categoryData_Engineer  0.27136    0.07269    3.733 0.000215 ***
## job_categoryData_Scientist  0.30198    0.07295    4.139 4.19e-05 ***
## job_categoryManagement_Lead  0.43254    0.09922    4.359 1.63e-05 ***
## job_categoryML_AI_Engineer  0.37474    0.08708    4.303 2.08e-05 ***
## job_categoryOther  0.14805    0.16007    0.925 0.355531
## job_categoryResearch  0.48002    0.14913    3.219 0.001384 **
## employment_typeCT  0.25834    0.22697    1.138 0.255661
## employment_typeFL -1.30607    0.29129   -4.484 9.42e-06 ***
## employment_typePT -1.18283    0.19862   -5.955 5.40e-09 ***
## company_sizeM  0.19186    0.07451    2.575 0.010362 *
## company_sizeL  0.31645    0.07560    4.186 3.45e-05 ***
## location_groupCA  0.68055    0.12502    5.444 8.79e-08 ***
## location_groupDE  0.56951    0.13318    4.276 2.34e-05 ***
## location_groupES  0.12064    0.17210    0.701 0.483666
## location_groupFR  0.33567    0.15557    2.158 0.031501 *
## location_groupGB  0.54654    0.10589    5.161 3.76e-07 ***
## location_groupIN -0.66423    0.12321   -5.391 1.16e-07 ***
## location_groupUS  0.96275    0.07381   13.044 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4967 on 430 degrees of freedom
## Multiple R-squared:  0.6373, Adjusted R-squared:  0.6196
## F-statistic: 35.98 on 21 and 430 DF,  p-value: < 2.2e-16
```

```
cat("\nTerms dropped (full model -> stepwise model):",
    setdiff(attr(terms(ols_full), "term.labels"),
            attr(terms(ols_step), "term.labels")), "\n")
```

```
##
## Terms dropped (full model -> stepwise model): remote_ratio work_year
```

```
tibble(
  model = c("Full OLS", "Stepwise-reduced OLS"),
  AIC = c(AIC(ols_full), AIC(ols_step)),
  adj_R2 = c(summary(ols_full)$adj.r.squared, summary(ols_step)$adj.r.squared),
  n_terms = c(length(attr(terms(ols_full), "term.labels")),
              length(attr(terms(ols_step), "term.labels")))
)
```

model	AIC	adj_R2	n_terms
Full OLS	676.3533	0.6188339	7
Stepwise-reduced OLS	673.5373	0.6196116	5

Robust inference under unequal error variances

Unequal error variances invalidate ordinary standard errors, so the coefficient table below is accompanied by HC1-robust (heteroskedasticity-consistent) ones:

```
robust_se <- lmtest::coefest(ols_step, vcov = sandwich::vcovHC(ols_step, type = "HC1"))
cat("HC1-robust inference for the stepwise-selected ln(salary) model:\n")
```

```
## HC1-robust inference for the stepwise-selected ln(salary) model:
```

```
print(robust_se)
```

```
##
## t test of coefficients:
##
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    9.938106   0.134761  73.7463 < 2.2e-16 ***
## experience_levelMI  0.215117   0.087010   2.4723  0.013810 *
## experience_levelSE  0.517240   0.081983   6.3091 6.985e-10 ***
## experience_levelEX  0.953145   0.119672   7.9646 1.496e-14 ***
## job_categoryData_Engineer  0.271361   0.063506   4.2730 2.376e-05 ***
## job_categoryData_Scientist  0.301985   0.067284   4.4882 9.234e-06 ***
## job_categoryManagement_Lead  0.432541   0.083675   5.1693 3.606e-07 ***
## job_categoryML_AI_Engineer  0.374737   0.092387   4.0562 5.924e-05 ***
## job_categoryOther    0.148053   0.144589   1.0240  0.306431
## job_categoryResearch  0.480016   0.170498   2.8154  0.005096 **
## employment_typeCT    0.258341   0.210629   1.2265  0.220673
## employment_typeFL   -1.306068   0.252194  -5.1788 3.437e-07 ***
## employment_typePT   -1.182832   0.262477  -4.5064 8.509e-06 ***
## company_sizeM        0.191864   0.096520   1.9878  0.047465 *
## company_sizeL        0.316454   0.097208   3.2554  0.001222 **
## location_groupCA     0.680548   0.123193   5.5242 5.741e-08 ***
## location_groupDE     0.569507   0.137779   4.1335 4.297e-05 ***
## location_groupES     0.120645   0.140063   0.8614  0.389518
## location_groupFR     0.335672   0.135341   2.4802  0.013513 *
## location_groupGB     0.546537   0.119574   4.5707 6.361e-06 ***
## location_groupIN    -0.664231   0.147050  -4.5170 8.112e-06 ***
## location_groupUS     0.962754   0.110046   8.7487 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The robust standard errors leave the significance picture essentially unchanged: every driver flagged as significant below remains significant under the robust (HC1) errors, so the substantive conclusions are not an artifact of unequal error variances.

What drives salaries?

Because the response is a logarithm, each coefficient translates into an approximate percentage change in salary through $100(e^\beta - 1)$:

```
coef_pct <- broom::tidy(ols_step) %>%
  mutate(pct_effect = 100 * (exp(estimate) - 1)) %>%
  dplyr::select(term, estimate, p.value, pct_effect)
print(coef_pct, n = 30)
```

```
## # A tibble: 22 x 4
##   term                estimate  p.value pct_effect
##   <chr>              <dbl>    <dbl>    <dbl>
## 1 (Intercept)        9.94  6.45e-288 2070350.
## 2 experience_levelMI  0.215 4.45e- 3    24.0
## 3 experience_levelSE  0.517 9.72e-11    67.7
## 4 experience_levelEX  0.953 9.62e-12   159.
## 5 job_categoryData_Engineer 0.271 2.15e- 4    31.2
## 6 job_categoryData_Scientist 0.302 4.19e- 5    35.3
## 7 job_categoryManagement_Lead 0.433 1.63e- 5    54.1
## 8 job_categoryML_AI_Engineer 0.375 2.08e- 5    45.5
## 9 job_categoryOther    0.148 3.56e- 1    16.0
##10 job_categoryResearch  0.480 1.38e- 3    61.6
##11 employment_typeCT    0.258 2.56e- 1    29.5
##12 employment_typeFL   -1.31 9.42e- 6   -72.9
##13 employment_typePT   -1.18 5.40e- 9   -69.4
##14 company_sizeM        0.192 1.04e- 2    21.2
##15 company_sizeL        0.316 3.45e- 5    37.2
##16 location_groupCA     0.681 8.79e- 8    97.5
##17 location_groupDE     0.570 2.34e- 5    76.7
##18 location_groupES     0.121 4.84e- 1    12.8
##19 location_groupFR     0.336 3.15e- 2    39.9
##20 location_groupGB     0.547 3.76e- 7    72.7
##21 location_groupIN    -0.664 1.16e- 7   -48.5
##22 location_groupUS     0.963 5.31e-33   162.
```

Four patterns stand out.

Experience dominates. Relative to an entry-level professional, mid-level colleagues earn about 24% more, seniors about 68% more, and executives about 159% more — a steep, strictly increasing gradient that makes seniority the single strongest career-side lever in the data.

Role matters within the field. Against the Data-Analyst baseline, moving into engineering (+31%) or science (+35%) pays moderately; specializing in machine learning/AI (+46%), leading teams (+54%), or research (+62%) pays substantially more. The heterogeneous “Other” category is statistically indistinguishable from analysts, which supports keeping it separate rather than forcing it into another family.

Contract structure shapes pay. Part-time (−69%) and freelance (−73%) arrangements earn far less than full-time contracts, consistent with partial-year or project-based engagement; contract work shows a positive but unreliable premium (+30%), unsurprising given only five such records.

Geography sets the level. Holding profession constant, a US-based employer pays roughly 162% above the rest-of-world baseline — by far the largest single effect in the model — with Canada (+98%), Great Britain (+73%), Germany (+77%) and France (+40%) forming a clear second tier, while India-based employers pay about 48% below it. Spain is indistinguishable from the baseline. Company size matters less: large firms

pay about 37% more than small ones ($p < 0.001$); medium firms sit at +21%, significant on both ordinary and robust errors ($p = 0.010$ ordinary, $p = 0.048$ robust).

4.3 Polynomial Regression

Stepwise selection suggested the linear trend terms added little, but that raises a subtler question: is the relationship curved? Polynomial regression answers it by allowing `remote_ratio` and `work_year` each to bend instead of following a straight line. Both variables take exactly three distinct values (0/50/100 and 2020/2021/2022), so degree 3 is not identifiable; degree 2 is the ceiling and effectively lets every level carry its own effect. We compare the linear and quadratic specifications with the nested F -test.

```
base_terms <- setdiff(attr(terms(ols_step), "term.labels"), c("remote_ratio", "work_year"))
base_formula_str <- paste("log_salary ~", paste(base_terms, collapse = " + "))

fit_poly <- function(degree) {
  f <- as.formula(paste0(base_formula_str,
                        " + poly(remote_ratio, ", degree, ")",
                        " + poly(work_year, ", degree, ")))
  lm(f, data = train)
}

poly_linear <- fit_poly(1) # straight-line benchmark
poly_quadratic <- fit_poly(2) # quadratic alternative

tibble(
  degree = c(1, 2),
  AIC = c(AIC(poly_linear), AIC(poly_quadratic)),
  adj_R2 = c(summary(poly_linear)$adj.r.squared, summary(poly_quadratic)$adj.r.squared)
)
```

degree	AIC	adj_R2
1	676.3533	0.6188339
2	672.6342	0.6235288

```
anova_poly <- anova(poly_linear, poly_quadratic)
anova_poly
```

Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
428	105.7992	NA	NA	NA	NA
426	104.0077	2	1.791456	3.668768	0.0263173

```
poly_model <- if (anova_poly$`Pr(>F)`[2] < 0.05) poly_quadratic else poly_linear

reset_result <- lmtest::resettest(poly_model)
cat("\nRESET test on selected model: F =", round(reset_result$statistic, 3),
    " p =", signif(reset_result$p.value, 4), "\n")
```

```
##
## RESET test on selected model: F = 2.081 p = 0.1261
```

The quadratic specification is preferred: the F -test rejects the purely linear form ($F = 3.67$, $p = 0.026$), adjusted R^2 edges up from 0.619 to 0.624, and the RESET test finds no remaining functional-form misspecification ($F = 2.08$, $p = 0.13$). In substantive terms, fully-remote and fully on-site arrangements sit at different pay levels than a straight line through the middle would suggest — the 50% hybrid point behaves differently from the average of the extremes.

4.4 Ridge and LASSO Regression

Ordinary least squares treats all twenty columns as free parameters. Shrinkage methods trade a little bias for variance reduction and, in the LASSO's case, automatic variable selection — a useful cross-check on the stepwise route. They require standardized inputs: all four numeric predictors are z-scored with training means and standard deviations, applied unchanged to the test records.

```
## Training-only z-scaling of every numeric input; the identical transform
## is applied to test.
tr_scale <- function(col) {
  m <- mean(train[[col]]); s <- sd(train[[col]])
  list(train = (train[[col]] - m) / s, test = (test[[col]] - m) / s)
}
for (col in c("exp_level_ord", "company_size_ord", "remote_ratio", "work_year")) {
  sc <- tr_scale(col)
  train[[paste0(col, "_ts")]] <- sc$train
  test[[paste0(col, "_ts")]] <- sc$test
}

build_matrix <- function(df) {
  model.matrix(
    ~ exp_level_ord_ts + company_size_ord_ts + remote_ratio_ts +
      work_year_ts + job_category + employment_type + location_group,
    data = df
  )[, -1] # glmnet supplies its own intercept
}

x_train <- build_matrix(train)
x_test <- build_matrix(test)
y_train <- train$log_salary
y_test <- test$log_salary
dim(x_train); dim(x_test)

## [1] 452 20

## [1] 113 20
```

The penalty strength is tuned by ten-fold cross-validation within the training set; prediction uses `lambda.min`, with the sparser `lambda.1se` reported for reference.

```
set.seed(2026)
cv_ridge <- cv.glmnet(x_train, y_train, alpha = 0, nfolds = 10)
set.seed(2026)
cv_lasso <- cv.glmnet(x_train, y_train, alpha = 1, nfolds = 10)

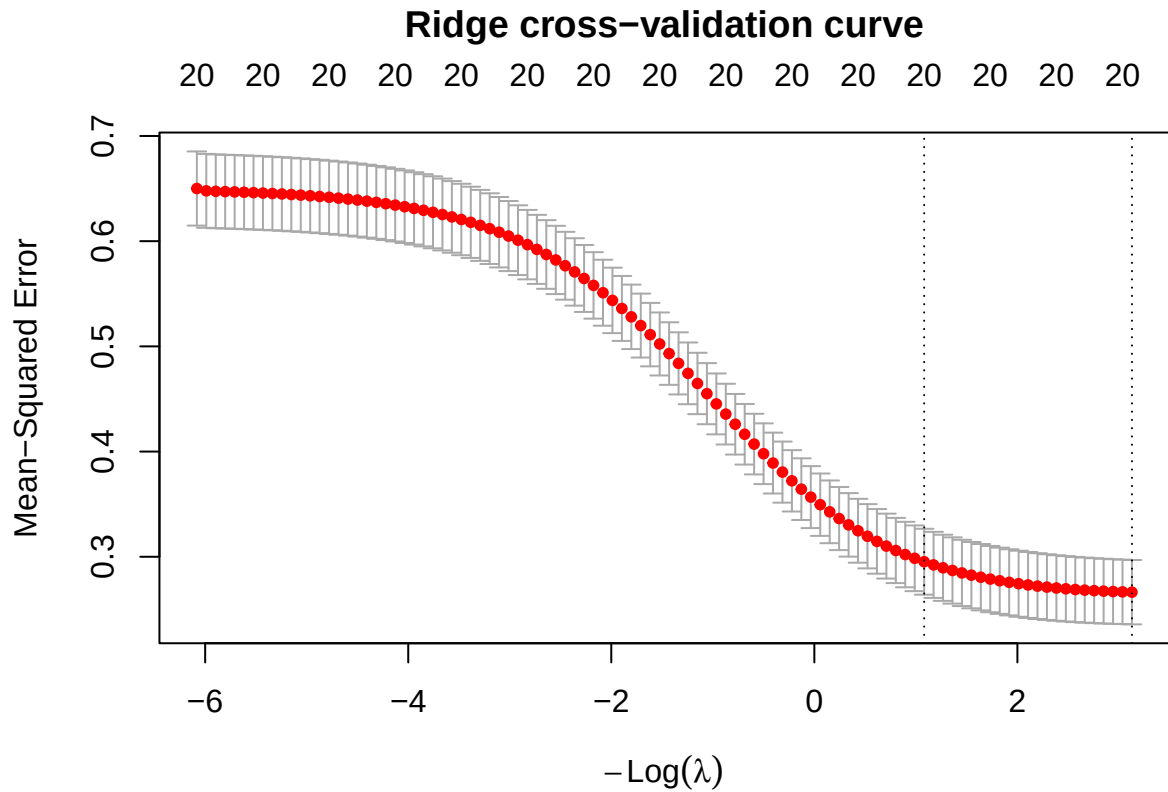
cat("Ridge: lambda.min =", cv_ridge$lambda.min, " lambda.1se =", cv_ridge$lambda.1se, "\n")
```

```
## Ridge: lambda.min = 0.04379278  lambda.1se = 0.3390716
```

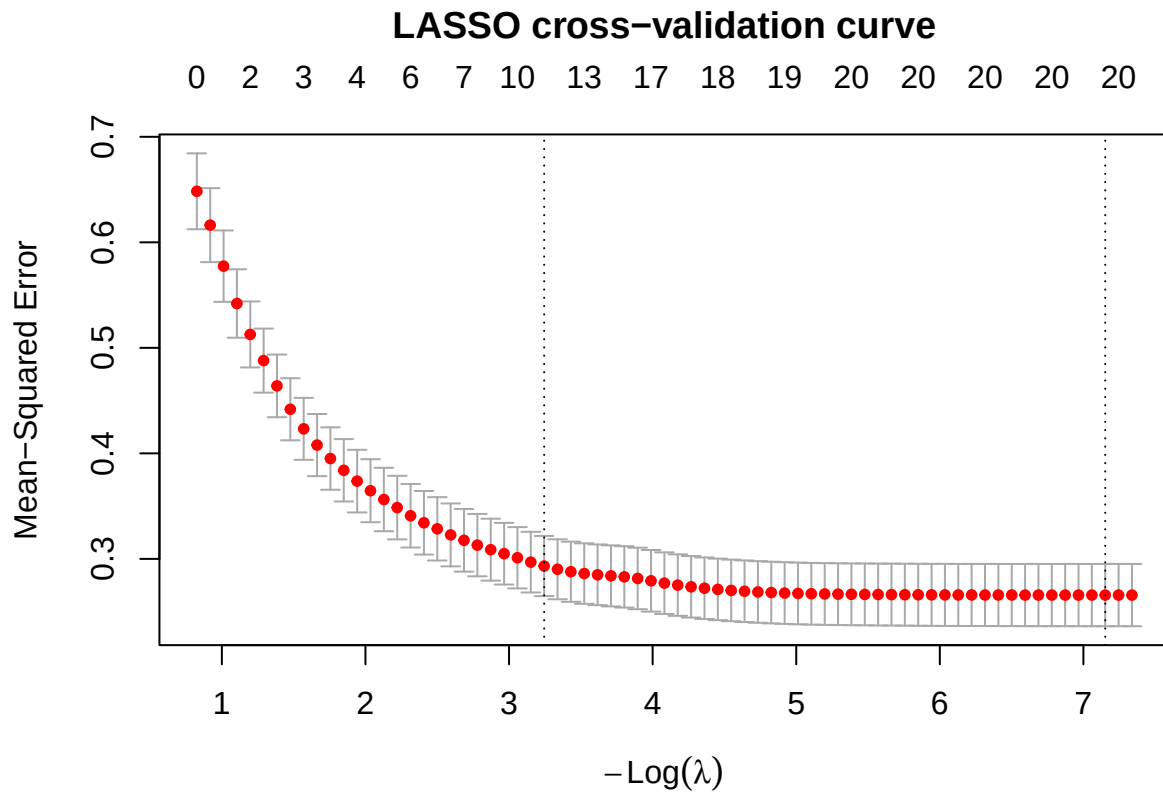
```
cat("LASSO: lambda.min =", cv_lasso$lambda.min, " lambda.1se =", cv_lasso$lambda.1se, "\n")
```

```
## LASSO: lambda.min = 0.0007832994  lambda.1se = 0.03898499
```

```
plot(cv_lasso); title("Ridge cross-validation curve", line = 2.5)
```



```
plot(cv_lasso); title("LASSO cross-validation curve", line = 2.5)
```

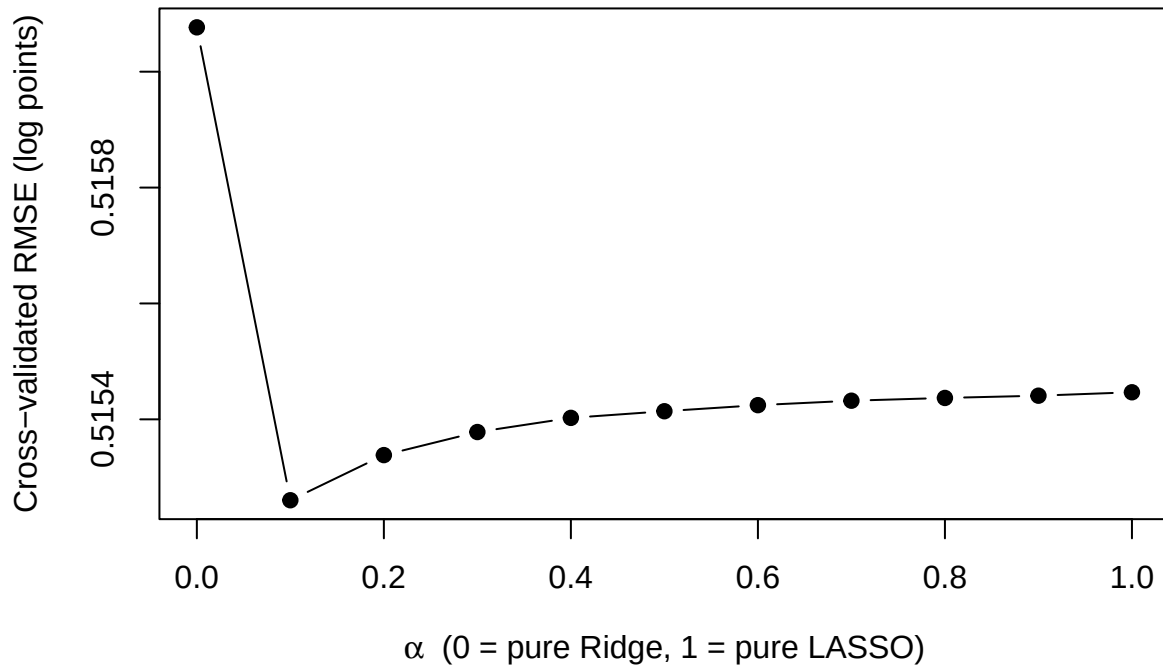
```
## Robustness of the endpoint choice: a sweep over the full elastic-net
## mixing range finds the CV surface flat, so neither pure penalty is a
## lucky guess.
```

```
enet_sweep <- vapply(seq(0, 1, by = 0.1), function(a) {
  set.seed(2026)
  sqrt(min(cv.glmnet(x_train, y_train, alpha = a, nfolds = 10)$cvm))
}, numeric(1))
cat("CV RMSE across alpha = 0,0.1,...,1:",
    sprintf("%.4f-%.4f (range %.4f)", min(enet_sweep), max(enet_sweep),
        diff(range(enet_sweep))), "\n")
```

```
## CV RMSE across alpha = 0,0.1,...,1: 0.5153-0.5161 (range 0.0008)
```

```
plot(seq(0, 1, by = 0.1), enet_sweep, type = "b", pch = 19,
     xlab = expression(alpha * " (0 = pure Ridge, 1 = pure LASSO)"),
     ylab = "Cross-validated RMSE (log points)",
     main = "Elastic-net mixing sweep")
```

Elastic-net mixing sweep



```
## Tuned elastic net: alpha picked by the same 10-fold CV, kept alongside its
## two pure-parent special cases for Task 6 to score.
alpha_best <- seq(0, 1, by = 0.1)[which.min(enet_sweep)]
set.seed(2026)
cv_elastic <- cv.glmnet(x_train, y_train, alpha = alpha_best, nfolds = 10)
cat("Selected elastic-net alpha:", alpha_best,
    "| predictor columns kept at lambda.min:",
    sum(as.numeric(coef(cv_elastic, s = "lambda.min"))[-1] != 0), "/", ncol(x_train), "\n")
```

```
## Selected elastic-net alpha: 0.1 | predictor columns kept at lambda.min: 20 / 20
```

Sweeping the mixing parameter across its full range ($\alpha = 0, 0.1, \dots, 1$) moves the cross-validated error by about 0.16%, and every mixture retains all twenty columns at $\lambda_{\min}$. With the surface this flat there is no meaningful interior optimum; the CV-selected mixture ($\alpha = 0.1$) is carried forward for Task 6 to score alongside the other specifications.

Which variables survive the LASSO penalty, and do they agree with the stepwise choice? Since LASSO names individual dummy columns while stepwise names whole variables, survivors are first mapped back to their parent variables before comparing.

```
lasso_coefs_min <- coef(cv_lasso, s = "lambda.min")
lasso_survivors <- setdiff(rownames(lasso_coefs_min)[which(lasso_coefs_min[, 1] != 0)],
    "(Intercept)")
cat("LASSO retains", length(lasso_survivors), "of", nrow(lasso_coefs_min) - 1,
    "columns at lambda.min\n")
```

```
## LASSO retains 20 of 20 columns at lambda.min
```

```
parent_terms <- c("exp_level_ord_ts", "company_size_ord_ts",
                  "remote_ratio_ts", "work_year_ts",
                  "job_category", "employment_type", "location_group")
map_to_parent <- function(dummy_names) {
  vapply(dummy_names, function(nm) {
    hit <- parent_terms[startsWith(nm, parent_terms)]
    if (length(hit) == 0) nm else hit[which.max(nchar(hit))]
  }, character(1), USE.NAMES = FALSE)
}
canonical <- c(exp_level_ord_ts      = "experience_level",
               company_size_ord_ts   = "company_size",
               remote_ratio_ts       = "remote_ratio",
               work_year_ts          = "work_year")
lasso_parents <- unique(map_to_parent(lasso_survivors))
lasso_parents <- unique(ifelse(lasso_parents %in% names(canonical),
                              canonical[lasso_parents], lasso_parents))
stepwise_terms <- attr(terms(ols_step), "term.labels")

cat("Selected by BOTH:", intersect(stepwise_terms, lasso_parents), "\n")
```

```
## Selected by BOTH: experience_level job_category employment_type company_size location_group
```

```
cat("Stepwise only:", setdiff(stepwise_terms, lasso_parents), "\n")
```

```
## Stepwise only:
```

```
cat("LASSO only:", setdiff(lasso_parents, stepwise_terms), "\n")
```

```
## LASSO only: remote_ratio work_year
```

The two selection philosophies converge on the same story: both retain the five substantive blocks — experience level, job family, contract type, company size, and location. LASSO additionally preserves small linear `remote_ratio` and `work_year` components that AIC pruning had discarded; their coefficients are negligible in size, so the disagreement is cosmetic rather than substantive.

A final sensitivity check replaces the ordinal encodings of experience and company size with full dummies inside the LASSO, the two variants being compared by ten-fold cross-validation on the training data. Accuracy hardly moves (CV RMSE 0.5154 vs 0.5176): the ordinal simplification costs nothing predictive.

```
build_matrix_factor <- function(df) {
  model.matrix(
    ~ experience_level + company_size + remote_ratio_ts + work_year_ts +
      job_category + employment_type + location_group,
    data = df
  )[, -1]
}
x_train_f <- build_matrix_factor(train)

set.seed(2026)
cv_lasso_factor <- cv.glmnet(x_train_f, y_train, alpha = 1, nfolds = 10)

cat("LASSO CV RMSE (log scale, training folds):\n")
```

```
## LASSO CV RMSE (log scale, training folds):
```

```
cat("  ordinal encoding:", round(sqrt(min(cv_lasso$cvm)), 4),
    "| factor-dummy encoding:", round(sqrt(min(cv_lasso_factor$cvm)), 4), "\n")
```

```
##   ordinal encoding: 0.5154 | factor-dummy encoding: 0.5176
```

4.5 Summary of training-set fit

This task constructs the candidate models, verifies their assumptions, and interprets the drivers on the training data; formal scoring of the held-out records and the final model choice belong to Task 6. To close the modeling here, the table below summarizes how well each specification fits the 452 training rows. Because every model except the power transform shares `log_salary` as its response, their R^2 and adjusted R^2 are directly comparable on the log scale. AIC — which rewards fit while penalizing parameter count — is reported for the three ordinary least-squares fits where it is defined; it is not defined for the penalty-regularized Ridge/LASSO/elastic-net fits, whose effective parameter count is not the number of columns.

```
pred_tr <- list(
  `Full OLS - stepwise`      = predict(ols_step),
  `Full OLS`                = predict(ols_full),
  `Polynomial regression`    = predict(poly_model),
  `Ridge regression`        = as.numeric(predict(cv_ride, newx = x_train, s = "lambda.min")),
  `LASSO regression`        = as.numeric(predict(cv_lasso, newx = x_train, s = "lambda.min")),
  `Elastic net regression`   = as.numeric(predict(cv_elastic, newx = x_train, s = "lambda.min"))
)

fit_row <- function(model, fit, y, n_coef, aic_val) {
  n      <- length(y)
  rss    <- sum((y - fit)^2)
  r2     <- 1 - rss / sum((y - mean(y))^2)
  adj    <- 1 - (1 - r2) * (n - 1) / (n - n_coef - 1)
  tibble(model = model,
    R2      = signif(r2, 4),
    adj_R2  = signif(adj, 4),
    n_par   = n_coef,
    AIC     = aic_val)
}

p_full <- length(coef(ols_full))
p_step <- length(coef(ols_step))
p_poly <- length(coef(poly_model))

train_fit <- bind_rows(
  fit_row("Multiple linear regression - full",      pred_tr[["Full OLS"]],      y_train, p_full, 1),
  fit_row("Multiple linear regression - stepwise",  pred_tr[["Full OLS - stepwise"]], y_train, p_step, 1),
  fit_row("Polynomial regression",                  pred_tr[["Polynomial regression"]], y_train, p_poly, 1),
  fit_row("Ridge regression",                        pred_tr[["Ridge regression"]],    y_train, ncol(x_train), 1),
  fit_row("LASSO regression",                        pred_tr[["LASSO regression"]],    y_train, ncol(x_train), 1),
  fit_row("Elastic net regression",                  pred_tr[["Elastic net regression"]], y_train, ncol(x_train), 1)
)

train_fit
```

model	R2	adj_R2	n_par	AIC
Multiple linear regression - full	0.6383	0.6179	24	676.4
Multiple linear regression - stepwise	0.6373	0.6187	22	673.5
Polynomial regression	0.6444	0.6226	26	672.6
Ridge regression	0.6320	0.6149	20	NA
LASSO regression	0.6361	0.6192	20	NA
Elastic net regression	0.6359	0.6190	20	NA

```
cat("\nNote: the Box-Cox power-scale model (Section 4.2) is fitted on\n")
```

```
##
```

```
## Note: the Box-Cox power-scale model (Section 4.2) is fitted on
```

```
cat("a different response scale, so its in-sample R2 is not directly\n")
```

```
## a different response scale, so its in-sample R2 is not directly
```

```
cat("comparable with the log-salary models above and is omitted here; the\n")
```

```
## comparable with the log-salary models above and is omitted here; the
```

```
cat("held-out comparison across scales is deferred to Task 6.\n")
```

```
## held-out comparison across scales is deferred to Task 6.
```

Three points follow. First, the **stepwise-reduced model matches the full model on training data** — equal adjusted R^2 (0.619) at a slightly lower AIC (674 vs 676) with two fewer parameters — so the pruning costs no in-sample fit while removing the marginal linear trend terms, the qualitative conclusion of Section 4.2. Second, the **regularized fits show essentially identical training R^2 to the OLS variants** (all in the range 0.63–0.64), as expected: shrinkage buys stability rather than in-sample fit, and their advantage, if any, materializes out of sample — which Task 6 measures. Third, **adjusted R^2 falls below R^2 by only a small margin** (about 0.02), because the models’ predictor count is modest relative to 452 training rows, so none of the candidates is badly over-parameterized in sample. In short, on the training data the candidate models are nearly indistinguishable in fit; their separation must come from held-out performance, which Task 6 evaluates.

```
write.csv(train, "dataset/train_data.csv", row.names = FALSE)
write.csv(test, "dataset/test_data.csv", row.names = FALSE)
```